

MimicAgent: Learning Quadruped Skills via Text-to-Trajectory Generation

Narayanan Palghat Parameswaran^{*1} Lucky Kant Nayak^{*1} Neehar Peri¹ Deva Ramanan¹

Abstract

We present MimicAgent, a text-to-trajectory generation framework for learning dynamic quadruped skills. Although reward shaping is extensively used when training quadruped policies, navigating the resulting reward landscape is notoriously difficult, requiring hours of "graduate student descent". Eureka attempts to automate reward design with LLMs, but we find that it struggles to generalize across diverse skills and morphologies. Motivated by the success of example-guided RL for humanoids, we revisit skill learning from demonstrations for quadrupeds. Unlike humanoids, which can exploit large-scale motion capture datasets for learning, quadrupeds lack such reference motion data. We make the observation that manually keyframing quadruped reference motions can be more intuitive than reward shaping; in particular, we find that rather coarse and even dynamically-infeasible motions can still be effective reference targets for example-guided RL. However, manual keyframing is still too cumbersome to create large-scale skill libraries. To address this challenge, we propose an LLM-based pipeline that generates kinematically feasible quadruped trajectories for diverse skills. Although these trajectories are not dynamically feasible, we show that they are sufficient to train successful policies. Across all evaluated skills, human raters consistently prefer policies generated by MimicAgent over those produced by Eureka. See our [project page](#) for more details.

1. Introduction

In recent years, learning-based locomotion policies have demonstrated impressive agility and robustness across a wide range of behaviors (Mahankali et al., 2024; Margolis & Agrawal, 2023) and terrains (Lee et al., 2020). However, training such RL policies typically relies on carefully tuned

¹Carnegie Mellon University. Correspondence to: Narayanan Palghat Parameswaran <npalghat@andrew.cmu.edu>.

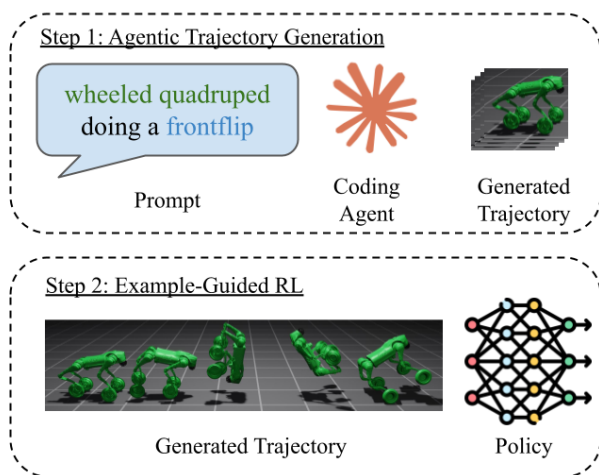


Figure 1. **Overview.** MimicAgent uses LLM coding agents to generate coarse (and even dynamically-infeasible) trajectories from text prompts (Step 1). We show that these coarse motions are effective reference targets for example-guided RL, and that generating plausible trajectories with an LLM is often easier than designing robust reward functions for the same behaviors (Step 2).

rewards. For complex and highly dynamic skills, designing bespoke reward functions is notoriously difficult and often requires extensive trial-and-error (Kumar et al., 2021; Nahrendra et al., 2023; Long et al., 2023). This challenge has motivated recent efforts to eliminate manual reward engineering entirely.

Automating Reward Design with LLMs. Recent work has explored using LLMs to automate reward design (Yu et al., 2023; Ma et al., 2023). For example, Eureka leverages coding LLMs to generate and iteratively refine reward functions from natural language task descriptions. While promising, these methods still depend on hand-engineered success metrics to evaluate candidate rewards and often struggle to generalize across diverse skills and morphologies. In practice, generated reward functions are brittle and often don't accurately follow natural language guidance, particularly for underexplored robot embodiments such as wheeled quadrupeds (Table 1).

Learning from Examples. An alternative to reward-centric RL is learning from demonstrations. Example-guided RL has been a key enabler for recent progress in humanoid locomotion, where motion capture datasets (Mahmood et al.,

2019; Allshire et al., 2025; Xu et al., 2025; He et al., 2024; Ji et al., 2024; Yu et al., 2021; Luo et al., 2023; Peng et al., 2018b), or manually annotated keyframes (Peng et al., 2018a; 2021; Zhang et al., 2025) provide rich supervision. However, quadrupeds lack such large-scale, publicly available datasets. Recent works (Bellegarda et al., 2025; Tirumala et al., 2019) curate libraries of simple quadruped motion patterns like walking, trotting, and galloping. However, such efforts are orders of magnitude smaller and less diverse than comparable humanoid datasets. This data scarcity limits the applicability of demonstration-based methods despite their strong empirical performance.

Agentic Trajectory Generation. To address this gap, we propose MimicAgent, an LLM-based agentic pipeline that directly generates text-guided example trajectories for policy learning. *Our key observation is that it is far easier for a human – and by association, an LLM – to generate reference motions than to generate reward functions.* Given a natural language skill description, MimicAgent produces executable code defining base motion and foot trajectories. This code is then executed in a MuJoCo engine using forward kinematics to produce kinematically feasible trajectories, without running a full physics simulator. Although the resulting trajectories are not dynamically feasible, they provide structured, task-relevant supervision that can be consumed by a downstream imitation-based RL pipeline like DeepMimic (Peng et al., 2018a).

Contributions. We present three major contributions. First, we introduce MimicAgent, an LLM-based text-to-trajectory framework for learning dynamic quadruped skills. Although MimicAgent generates dynamically infeasible trajectories, we show that they are sufficient to train successful policies. Our experiments highlight the limitations of prior LLM-based approaches for reward shaping, and demonstrate the effectiveness of using LLMs for trajectory synthesis.

2. Related Works

Quadrupedal Locomotion has matured significantly in recent years (Kumar et al., 2021; Lee et al., 2020; Nahrendra et al., 2023; Margolis & Agrawal, 2023; Mahankali et al., 2024; Smith et al., 2023). Early work primarily focused on training legged robots to execute complex contact sequences to achieve different locomotion gaits. A common approach is to use model predictive control (MPC) to track predefined contact patterns (Grandia et al., 2019; Villarreal et al., 2020), enabling canonical gaits such as trotting (Di Carlo et al., 2018), pacing (Raibert, 1990), bounding (Eckert et al., 2015), and galloping (Marhefka et al., 2003), as well as non-conventional gaits with customized contact timings (Li et al., 2022; Winkler et al., 2018). Despite their success, MPC-based approaches require hand-designed reference trajectories and incur high computational costs, limiting be-

havioral diversity. Recent work extends these ideas to agile bipedal quadrupedal motions (Li et al., 2024) and whole-body loco-manipulation (Qiu et al., 2025; Ouyang et al., 2025). In contrast, our approach leverages example-guided reinforcement learning, which can be more intuitive than designing foot contact sequences.

Text-to-Robot Control is an active area of research for both quadrupeds (Tang et al., 2023; Zhou et al., 2025; Jian et al., 2025) and humanoids (Cui et al., 2024; Tessler et al., 2023; Cui et al., 2025; Tirinzoni et al., 2025). Early methods relied on structured text templates or NLP tools like parse trees to extract constraints and generate robot trajectories (Kress-Gazit et al., 2008; Chai et al., 2018; Howard et al., 2014). Other approaches used representation learning to train language-conditioned policies that map free-form instructions directly to actions (Stepputtis et al., 2020; Brohan et al., 2022; Mees et al., 2022; Shridhar et al., 2022). However, such approaches require large annotated datasets with paired text and robot control, which is difficult to collect for diverse locomotion behaviors. More recent methods prompt LLMs to generate robot code, bridging the gap between language and motor control via intermediate representations like high-level plans, primitive skills, or trajectories (Vempala et al., 2024; Lin et al., 2023; Liang et al., 2022; Singh et al., 2022; Bucker et al., 2022). Inspired by prior work, we leverage LLMs to directly generate reference trajectories for imitation learning.

3. MimicAgent: Learning Dynamic Skills with LLM-based Trajectory Synthesis

MimicAgent aims to automate motion imitation learning by replacing manual reward engineering with an agentic motion synthesis pipeline. Given a natural language description of a quadruped skill (e.g. `wheeled quadruped doing a frontflip`), MimicAgent generates reference motion trajectories that can be directly used by imitation-based RL algorithms. Rather than navigating a complex and brittle reward landscape, MimicAgent leverages the observation that generating a plausible reference trajectory is often significantly easier than designing a robust reward function for the same behavior.

Agentic Motion Synthesis Pipeline. MimicAgent is an iterative pipeline that decomposes motion synthesis into *motion planning*, *code generation*, *execution*, and *refinement* (Algorithm 1).

The input to the system is a natural language skill description, which is processed by the motion planner agent to produce a phase-based motion plan encoding the temporal and kinematic structure of the skill over a cyclic phase variable. Each motion plan describes the overall motion intent and stability strategy, the decomposition into sub-phases with

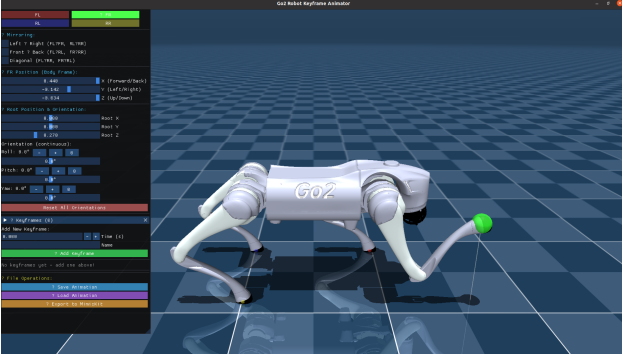


Figure 2. Keyframing Reference Motion. We visualize a MuJoCo forward kinematics model for *puppeteering* a quadruped by manually keyframing reference trajectories. We demonstrate that such an interface is easier for both humans and LLMs to tune, compared to standard workflows for reward shaping.

associated leg contact states, coordination patterns across leg groups, base linear and angular velocity commands in the body frame, and qualitative leg trajectories relative to the body. It also provides handoff notes for the code generation agent for controller implementation.

Each skill is represented over a normalized phase variable $\phi \in [0, 1]$, allowing motions of arbitrary duration to be expressed in a unified, phase-indexed form. A trajectory is defined as a sequence of keyframes sampled at discrete phase values. Each keyframe specifies the base position $\mathbf{p}(t)$, base orientation $\mathbf{q}(t)$, and joint rotations $\boldsymbol{\theta}(t)$.

$$\mathbf{p}_t \in \mathbb{R}^3, \quad \mathbf{q}_t \in \mathbb{R}^4, \quad \|\mathbf{q}_t\| = 1, \quad \boldsymbol{\theta}_t \in \mathbb{R}^n$$

$$\mathbf{X}_{0:T} = \left\{ \mathbf{x}_t = \begin{bmatrix} \mathbf{p}_t \\ \mathbf{q}_t \\ \boldsymbol{\theta}_t \end{bmatrix} \middle| t = 0, \dots, T \right\}$$

This representation enables a compact and interpretable parameterization capable of expressing a wide range of locomotion behaviors.

Code Synthesis and Trajectory Execution. Given the motion plan, a coding agent synthesizes an executable program that defines the temporal functions and sequence for base and foot trajectories. The generated code is executed in a MuJoCo simulator. Importantly, we do not perform any physics simulation. As a result, the synthesized trajectories are kinematically consistent but generally violate dynamic constraints. MimicAgent explicitly embraces this trade-off, delegating dynamic feasibility to the imitation learning stage.

Task-Agnostic Unit Tests. To validate generated trajectories without introducing task-specific biases, MimicAgent employs a set of *task-agnostic unit tests*. Rather than optimizing task rewards, MimicAgent filters generated mo-

tions using task-agnostic constraints, enabling scalable reference trajectory generation without manually specifying task-specific success criteria. These tests are designed to reject physically implausible or degenerate trajectories while remaining broadly applicable across skills. Each unit test computes a scalar reward, and the aggregate score determines whether a trajectory passes validation. The unit tests include:

- **Base Height Envelope.** The base height is constrained to remain within a reasonable envelope relative to the ground. Heights below 0.1 meters or those exceeding twice the standing base height are penalized. This test returns a binary reward of +1 or -1.
- **Joint Violation Penalty.** Joint limit violations are penalized proportional to their frequency. The reward is computed as $r_{\text{joint}} = -1 \times N_{\text{violated}}$, where N_{violated} denotes the number of joints exceeding their limits.
- **Ground Penetration.** Foot-ground penetration is penalized based on severity. Trajectories in which all four feet penetrate the ground incur a large penalty, while partial penetration results in a smaller penalty. The reward is bounded between +1 and -1.
- **Feet Air-Time Constraint.** For non-aerial skills, at least one foot must remain in contact with the ground for half of the motion cycle. This constraint prevents degenerate motions that unrealistically leave the ground for extended durations.

Importantly, these unit tests do not encode task success, performance objectives, or stylistic preferences. They serve purely as structural validity checks, ensuring that generated trajectories are suitable as imitation references. In contrast, Eureka (Ma et al., 2023) relies heavily on carefully designed task-specific success functions to generate reward signals. Defining such success criteria is a time-intensive process and is often a significant bottleneck for learning large skill libraries. Motivated by this limitation, MimicAgent adopts a task-agnostic validation strategy that avoids task-specific success criteria entirely.

Iterative Refinement. If a trajectory fails validation, a diagnostic agent analyzes the failure in the context of the full system state, including the skill description, motion plan, generated code, and unit test outcomes. The diagnostic agent produces natural language feedback identifying likely failure modes. A code modification agent then revises the trajectory-generating program accordingly. This process iterates until a trajectory passes all unit tests or a fixed iteration budget is reached. Valid trajectories are returned for downstream imitation learning. The code modification agent can optionally also take human feedback to improve code generation quality.

Algorithm 1 MimicAgent Trajectory Generation

```

def mimic_agent(skill_description, N=4):
    iters = 0
    state = [skill_description]

    motion_plan = motion_planner(
        skill_description)
    state.append(motion_plan)

    skill_code = coder_agent(motion_plan,
        state)
    state.append(skill_code)

    while not pass and iters < N:
        trajectory = code_executor(
            skill_code)
        pass = unit_test(trajectory)
        state.append(result)

        if pass:
            return trajectory

    llm_feedback = motion_diagnosis(
        state)
    state.append(llm_feedback)

    skill_code = code_modifier(
        skill_code, state,
        human_feedback=None)
    iters += 1

    return None

```

By shifting the focus from reward optimization to reference trajectory synthesis, MimicAgent avoids the brittle and time-consuming process of manual reward engineering. Unlike LLM-based reward design methods that require repeated policy training and hand-crafted evaluation criteria, MimicAgent generates supervision directly in trajectory space. This makes the framework efficient, interpretable, and naturally extensible to diverse quadruped skills.

4. Experiments

In this section, we briefly describe our evaluation metrics, provide an empirical analysis of our baselines, and ablate the annotation time and impact of reference motion quality between human annotated keyframes and our agentic trajectory synthesis pipeline.

Taxonomy. We present a simple taxonomy of quadruped locomotion skills (Figure 3) that categorizes kinematic behaviors for wheeled quadruped robots according to the primary source of propulsion. The taxonomy consists of two main classes: *Active (Wheel-Driven)* and *Passive (Limb-Driven)* skills. Active skills are defined by wheel joint actuation as the dominant mechanism for locomotion; while

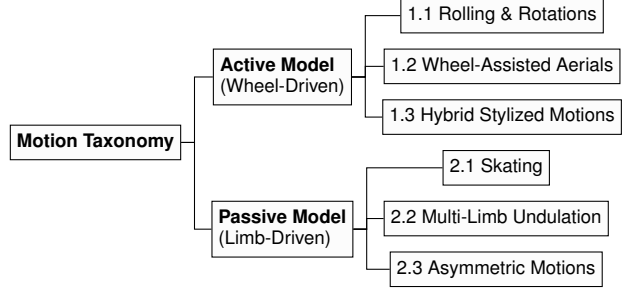


Figure 3. **Quadruped Taxonomy.** Kinematic locomotion skills categorized by primary source of propulsion: wheel-dominant (*Active*) vs. limb-dominant (*Passive*). Each skill is labeled with an ID (e.g., 1.1 for Rolling & Rotations) for easy reference.

leg joints may move freely and can exhibit large, coordinated motions for posture adjustment, stylization, or dynamic effects, they are not intended to contribute propulsive forces. This category includes planar rolling and wheel maneuvers like straight, curved, looping trajectories, pivots, and spins, wheel-assisted aerial behaviors in which wheels govern translational displacement while limbs shape the aerial pose, and hybrid stylized motions that combine rolling, rotation, or limb coordination to produce expressive, non-canonical trajectories. In contrast, passive skills rely exclusively on coordinated leg-joint actuation, with wheels remaining inactive such that propulsion emerges entirely from limb kinematics. Representative behaviors in this class include skating motions generated through lateral or diagonal leg sweeps, multi-limb undulation producing wave-like body or contact patterns, and asymmetric motions characterized by non-canonical limb sequencing, diagonal pushes, or alternating support patterns that produce complex trajectories. We use this taxonomy to propose skills below.

Metrics. We quantitatively compare manual keyframing, Eureka (Ma et al., 2023), and MimicAgent with a user study. Participants were presented with pairs of textual task descriptions and videos of trained policies corresponding to that task. Participants were asked to rate how well the policy follows the task description on a 5-point Likert scale. Videos were presented in a randomized order to eliminate ordering bias and participants were not informed which method was used to generate each policy. In addition, we compute the mean squared error between the reference trajectories and the trajectories produced by the learned policy for both human annotated keyframes and MimicAgent’s keyframes to measure how faithfully the learned policy follows the demonstration. This allows us to measure the quality and dynamic feasibility of the reference motion.

Baselines. We compare MimicAgent with policies trained by Eureka (Ma et al., 2023) and human annotated keyframes. Eureka frames reward design as a program synthesis problem: given a textual skill description and access to environ-

ment code (e.g., observations, episode termination conditions, and success function), an LLM generates candidate reward functions. Multiple reward functions are sampled per iteration, trained in parallel, and evaluated using predefined success criteria, with an evolutionary search loop selecting and refining the best-performing rewards. Next, we also evaluate DeepMimic (Peng et al., 2018a) policies learned from human annotated keyframes. To simplify the annotation process, we created a custom annotation tool with Claude Sonnet 4.5 to control foot positions and root body pose using click-and-drag mechanics for coarse control and sliders for fine-grained control (Figure 2).

Motion Tracking Policy. Following (Peng et al., 2018a), we train a motion tracking policy $\pi(a_t | o_t)$ to imitate reference motions generated by our pipeline. At each timestep t , the policy receives an observation o_t and outputs an action a_t , which specifies desired joint targets for a low-level PD controller. The observation is composed of the current robot state and target reference state. The current observation encodes the configuration of the simulated robot at time t :

$$o_t^{\text{char}} = [\dot{\mathbf{p}}_t^{\text{root}}, \mathbf{q}_t^{\text{root}}, \boldsymbol{\omega}_t^{\text{root}}, \boldsymbol{\theta}_t, \dot{\boldsymbol{\theta}}_t, \mathbf{k}_t], \quad (1)$$

where $\dot{\mathbf{p}}_t^{\text{root}}$ denotes the root linear velocity, $\mathbf{q}_t^{\text{root}}$ and $\boldsymbol{\omega}_t^{\text{root}}$ represent the root orientation and angular velocity, respectively. The vectors $\boldsymbol{\theta}_t$ and $\dot{\boldsymbol{\theta}}_t$ correspond to joint positions and velocities, while $\mathbf{k}_t$ denotes the position of the key body (e.g. feet). In addition to the current state, the policy is conditioned on reference motion targets, which serve as control signals guiding the imitation process.

$$o_t^{\text{tar}} = [\mathbf{p}_t^{\text{root}}, \mathbf{q}_t^{\text{root}}, \boldsymbol{\theta}_t, \mathbf{k}_t], \quad (2)$$

The full observation is formed by concatenation:

$$o_t = [o_t^{\text{char}}, o_t^{\text{tar}}]. \quad (3)$$

Actions are parameterized as target joint rotations for a PD controller. The motion tracking reward consists of pose, velocity, root, and keypoint tracking terms:

$$r_t = w_p \exp\left(-\alpha_p \sum_j w_j^{\text{rot}} \|\Delta\boldsymbol{\theta}_t^j\|^2\right) \quad (4)$$

$$\begin{aligned} &+ w_v \exp\left(-\alpha_v \sum_j w_j^{\text{vel}} \|\Delta\dot{\boldsymbol{\theta}}_t^j\|^2\right) \\ &+ w_{rp} \exp(-\alpha_{rp} (\|\Delta\mathbf{p}_t^{\text{root}}\|^2 + 0.1 \|\Delta\mathbf{q}_t^{\text{root}}\|^2)) \\ &+ w_{rv} \exp(-\alpha_{rv} (\|\Delta\dot{\mathbf{p}}_t^{\text{root}}\|^2 + 0.1 \|\Delta\dot{\mathbf{q}}_t^{\text{root}}\|^2)) \\ &+ w_k \exp\left(-\alpha_k \sum_i \|\Delta\mathbf{k}_t^i\|^2\right), \end{aligned} \quad (5)$$

Table 1. Comparison to State-of-the-Art. We conduct a user study (n=30) to compare policies generated by human annotated keyframes, Eureka, and MimicAgent. We find that users prefer MimicAgent policies for Walking, Side Flip, Front Flip, and Aerial Crossover. Users prefer policies generated by human annotated keyframes for bounding, though all policies receive low ratings. Higher is better.

Skill	Walking	Bounding	Side Flip	Front Flip	Aerial Crossover
Robot	Go2	Go2	Go2-W	Go2-W	Go2-W
Key Frames	2.5	3.5	3.6	4.3	2.6
Eureka	1.8	2.8	1.3	1.4	2.2
MimicAgent	4.4	2.5	4.9	4.6	4.2

where $\Delta\boldsymbol{\theta}_t^j$ and $\Delta\dot{\boldsymbol{\theta}}_t^j$ denotes the joint angle and velocity difference between simulated and reference joint j , $\Delta\dot{\mathbf{q}}_t^j$ and $\Delta\dot{\boldsymbol{\omega}}_t^j$ denotes root orientation and angular velocity error, $\Delta\mathbf{p}_t^{\text{root}}$ and $\Delta\dot{\mathbf{p}}_t^{\text{root}}$ are root position and velocity errors, and $\Delta\mathbf{k}_t^i$ represents key body position errors expressed in the root frame. The policies are trained using PPO with reference state initialization and early termination as in DeepMimic.

Skills. We evaluate the following skills in our user study. Notably, these five skills are chosen because they are particularly easy to keyframe. We evaluate two additional skills that are significantly harder to keyframe (*Reverberating Yaw Pulse* and *Crab Diagonal Scuttle*) in the supplement.

- *UniTree Go2 Trot.* A symmetric diagonal gait where front-left and rear-right legs alternate with front-right and rear-left, maintaining continuous ground contact and a stable torso while tracking commanded forward velocity.
- *UniTree Go2 Bounding.* A dynamic gait where the front legs push off together followed by the rear legs, producing a brief aerial phase and pronounced vertical oscillation while maintaining forward propulsion.
- *UniTree Go2-W Front Flip.* An acrobatic maneuver where the robot generates strong forward pitch momentum, enters a fully airborne phase, rotates 360 about its lateral axis, and lands upright with controlled impact.
- *UniTree Go2-W Side Flip.* A lateral acrobatic motion where the robot initiates a rapid roll rotation about its longitudinal axis, becomes fully airborne, completes a full 360 side rotation, and recovers to a stable stance.
- *UniTree Go2-W Aerial Crossover.* While carving forward on its wheels, the robot lifts alternating diagonal leg pairs off the ground, executing rapid mid-air crossover wave motions.

Comparison with State-of-the-Art. We conduct a user study (n = 30) comparing policies generated from human annotated keyframes, Eureka, and MimicAgent. Participants

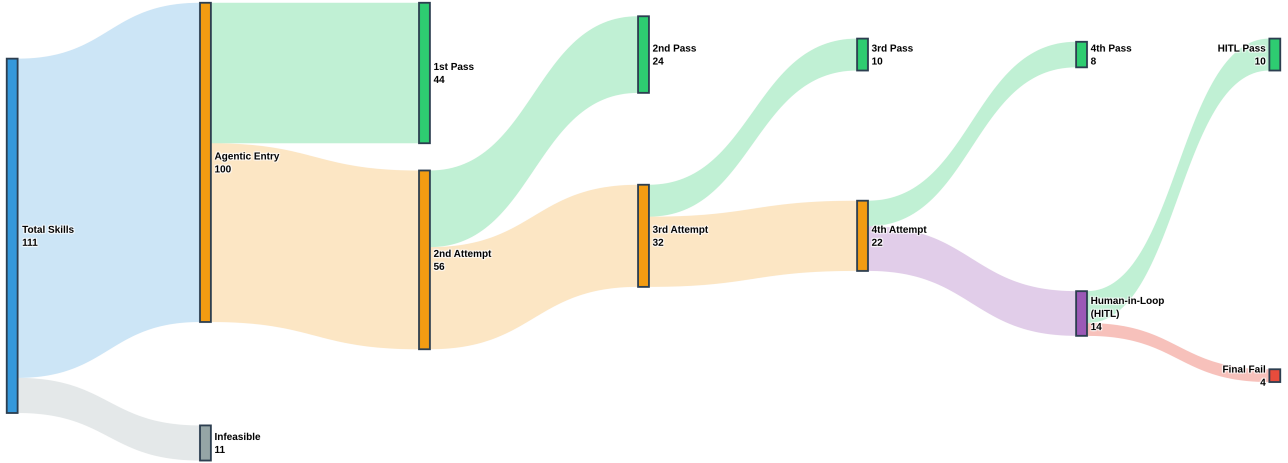


Figure 4. Automatic Skill Generation. We use an LLM to propose diverse locomotion skills according to our taxonomy, yielding 111 candidate skills. We generate trajectories for all skills using MimicAgent: 11 are deemed infeasible, while 78 succeed after up to four iterative attempts. Incorporating human-in-the-loop feedback further improves performance, increasing the total number of successful skills to 96. We visualize several examples in Figure 5.

Table 2. Comparison of Time-to-Annotation. We evaluate the time to annotation (in minutes) between human annotated keyframes and MimicAgent keyframes. Notably, MimicAgent is consistently faster than human annotated keyframes. Lower is better.

Skill	Walking	Bounding	Side Flip	Front Flip	Aerial Crossover
Robot	Go2	Go2	Go2-W	Go2-W	Go2-W
Key Frames	2.6	4.5	4.5	4.4	7.6
MimicAgent	1.1	1.2	2.9	3.2	3.6

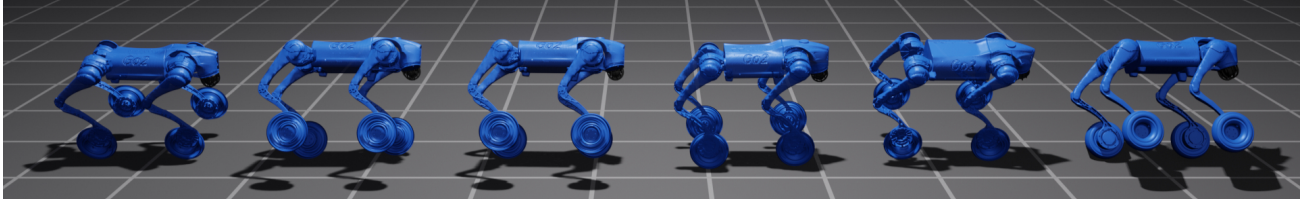
consistently prefer MimicAgent policies for Walking (+2.6 vs. Eureka), Side Flip (+3.6 vs. Eureka), Front Flip (+3.2 vs. Eureka), and Aerial Crossover (+2 vs. Eureka). For Bounding, users favor policies generated from human-annotated keyframes, although all approaches received relatively low ratings. Notably, Eureka still relies on carefully specified task-specific success metrics and exhibits limited robustness when scaling to diverse, stylistically different skills. Although MimicAgent does not require task-specific unit tests, we hand-engineer success criteria for Eureka’s policies to give it a better chance of succeeding. Despite this, performance degrades for wheeled quadruped skills (UniTree Go2-W), likely due to the increased complexity of the joint space. We provide videos for all policies used in this user study on our [project page](#).

Automatic Skill Generation. In addition to manually prompting MimicAgent, we can leverage LLMs to automatically generate large-scale skill libraries. Figure 4 visualizes the success rates of this automated skill generation process. We first prompt an LLM to propose candidate skills based on our taxonomy (Figure 3), resulting in 111 total skills. Of these, 11 are deemed infeasible, while 78

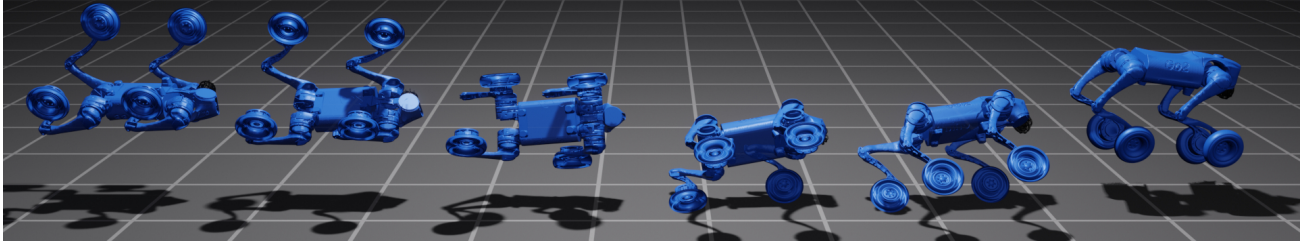
successfully converge after four iterations. We further show that incorporating non-expert human feedback improves policy performance, increasing the number of successful skills to 96. Notably, this feedback only provides high-level guidance rather than specific code modifications or hyper-parameter tuning. Success rate is determined by the number of skills that pass all unit tests.

Time-to-Annotation. We compare the time required for human-annotated keyframes and MimicAgent generated keyframes in Table 2. We find that MimicAgent consistently requires less time than manually annotated keyframes. The five skills evaluated are intentionally limited to relatively simple motions (e.g. unidirectional velocity or in-place yaw) and do not include more complex skills that combine these elements, such as asymmetric leg movements coupled with base rotations. Such skills are difficult and time-consuming to keyframe manually. In contrast, learning-based agents are able to generate these types of skills quickly (see supplement). We posit that as skill complexity increases, human annotation time will grow significantly faster than MimicAgent’s skill generation time.

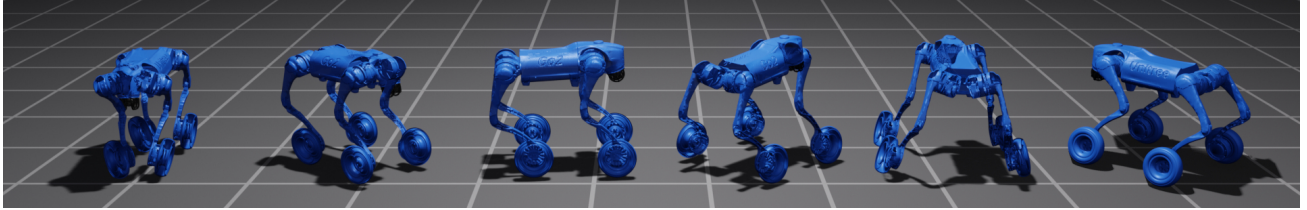
Comparison with Reference Motion. We compare DeepMimic-style policies trained on reference trajectories generated with our manual keyframing tool (133 frames) and MimicAgent (133 frames) in Table 3. Policies trained with MimicAgent reference trajectories consistently achieved lower error across all skills and evaluation metrics, suggesting that the original reference trajectories were more dynamically feasible than human-annotated trajectories. We track errors for root position in the global frame (E_{RP} , m), root rotation (E_{RR} , rad), joint rotation (E_{JR} , rad), joint



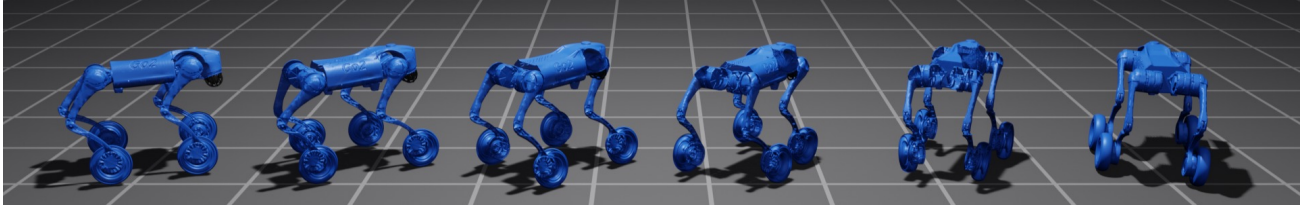
(a) Forward locomotion via large-amplitude lateral base swaying like an inverted pendulum. [1.3]



(b) Backward lateral locomotion with cartwheel-style leg rotations and circular trajectories. [1.2]



(c) Legs traces a diamond-shaped pattern on the ground. [1.1]



(d) Circular locomotion with side-to-side base swaying, like a swinging hammock. [2.2]

Figure 5. Qualitative Skill Synthesis Results. We present representative skills synthesized by MimicAgent across both active and passive motion regimes. Each subfigure shows a different skill (with the initial prompt in the subcaption) belonging to a certain taxonomy; please see the [project page](#) for trajectory rollouts.

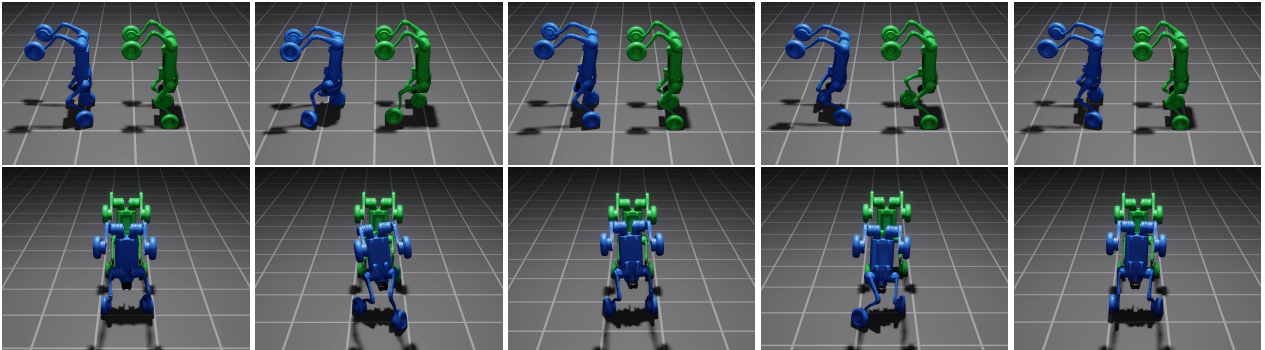


Figure 6. Generating Novel Skating Skills. MimicAgent can generate never-before-seen (to the best of our knowledge) quadruped skills, such as skating on underactuated wheels while balancing on two legs. Legged wheeled robots typically make use of actuated motors at the wheel, but humans can move on (freely rotating or under-actuated) roller skates by pushing off to the side. MimicAgent can generate keyframed reference motions (shown in green) that fail to capture such dynamics. However, such reference motions can nonetheless be used as a target for learning a RL policy that *does* capture such subtle dynamics (for the underactuated wheeled robot in blue).

Table 3. Comparison with Reference Motion. We evaluate DeepMimic policies trained with human annotated keyframes and MimicAgent keyframes against their respective reference trajectories. Across all skills and metrics, MimicAgent policies achieve lower mean squared error on both pose-level errors (root and body position and rotation), and motion-level errors (linear and angular velocities). Lower is better.

Method Robot	Error Metric	Walking Go2	Bounding Go2	Side Flip Go2-W	Front Flip Go2-W	Aerial Cross Go2-W
KeyFrame Tool	E_{RP}	0.170	0.076	0.199	0.099	0.020
	E_{RR}	0.177	0.208	0.238	0.208	0.025
	E_{JR}	0.360	0.320	0.484	0.507	0.396
	E_{JV}	4.313	3.868	8.380	15.183	6.823
	E_{RLV}	0.251	0.214	0.548	0.287	0.035
	E_{RAV}	0.918	0.517	1.883	1.126	0.233
MimicAgent	E_{RP}	0.052	0.049	0.077	0.089	0.013
	E_{RR}	0.046	0.114	0.207	0.160	0.024
	E_{JR}	0.109	0.224	0.382	0.528	0.373
	E_{JV}	0.646	1.580	5.299	12.871	4.761
	E_{RLV}	0.093	0.136	0.295	0.264	0.044
	E_{RAV}	0.176	0.353	1.869	1.697	0.211

velocity (E_{JV} , rad s⁻¹), root linear velocity (E_{RLV} , m s⁻¹), and root angular velocity (E_{RAV} , rad s⁻¹). This improvement is most pronounced for highly dynamic skills such as side flips, front flips, and aerial crossovers, where policies trained on manually keyframed trajectories exhibit significantly larger velocity and angular velocity errors. In contrast, policies trained on MimicAgent references result in improved motion timing and smoother rotational dynamics during policy learning. Even for simpler skills such as walking and bounding, MimicAgent yields lower global pose error, reflecting higher reference motion quality. These results indicate that improved reference trajectory quality directly reduces downstream imitation error, demonstrating the effectiveness of MimicAgent as a scalable alternative to manual keyframing for imitation-based RL.

Qualitative Results. We visualize policies trained with MimicAgent reference trajectories in Figure 5 and Figure 6. In Figure 5, we find that MimicAgent can successfully learn policies from all parts of our taxonomy (more videos on the [project page](#)). In Figure 6, we highlight that MimicAgent can generate (to the best of our knowledge) never-before-seen skating behavior. Although the reference trajectory does not capture the nuances of skating on underactuated wheels while balancing on two legs, training a policy with DeepMimic *does* capture such subtle dynamics.

Analysis of Failure Cases. Policies trained for jumping and midair 180° turns failed to learn the intended behaviors. Instead, the robot exploited the task by completing the turn on the ground, likely due to incorrect or near-static foot trajectories that did not sufficiently encourage lift-off. These issues align with recurring failure patterns observed

in practice, where agents become trapped in error loops (e.g., alternating between ground penetration and air penalties). This suggests a need for more specialized tool calls and verification checks, such as explicit validation of foot placement and contact conditions.

Limitations and Future Work. Unlike cyclic skills such as walking or skating, more complex, acyclic skills lack direct control over velocity commands or phase durations, making execution and debugging on real hardware difficult. This issue is exacerbated by largely task-agnostic unit tests; while they provide baseline coverage, we observe clear gains from human-in-the-loop feedback. This suggests that skill-specific feedback could further reduce failure rates and improve sim-to-real transfer. Future work should use LLMs to automatically generate skill-specific unit tests and provide targeted feedback during trajectory synthesis.

5. Conclusion

In this paper, we present MimicAgent, an LLM-based agentic pipeline for quadruped trajectory generation. By leveraging coding agents to synthesize coarse yet kinematically feasible reference motions, MimicAgent sidesteps the brittleness and manual effort of reward shaping while avoiding the need for large-scale motion capture datasets. Our results show that, despite being dynamically infeasible, these automatically generated trajectories are sufficient for training robust and expressive behaviors across a diverse set of skills. Human evaluations consistently favor policies trained with MimicAgent over those produced by Eureka, highlighting our method’s improved motion quality and language following abilities.

References

- Allshire, A., Choi, H., Zhang, J., McAllister, D., Zhang, A., Kim, C. M., Darrell, T., Abbeel, P., Malik, J., and Kanazawa, A. Visual imitation enables contextual humanoid control. *arXiv preprint arXiv:2505.03729*, 2025.
- Bellegarda, G., Shafiee, M., and Ijspeert, A. Allgaits: Learning all quadruped gaits and transitions. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 15929–15935. IEEE, 2025.
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- Bucker, A., Figueredo, L., Haddadin, S., Kapoor, A., Ma, S., Vemprala, S., and Bonatti, R. Latte: Language trajectory transformer. *arXiv preprint arXiv:2208.02918*, 2022.
- Chai, J. Y., Gao, Q., She, L., Yang, S., Saba-Sadiya, S., and Xu, G. Language to action: towards interactive task learning with physical agents. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence, IJCAI’18*, pp. 29. AAAI Press, 2018. ISBN 9780999241127.
- Cui, J., Liu, T., Liu, N., Yang, Y., Zhu, Y., and Huang, S. Anyskill: Learning open-vocabulary physical skill for interactive agents. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Cui, J., Liu, T., Meng, Z., Yu, J., Song, R., Zhang, W., Zhu, Y., and Huang, S. Grove: A generalized reward for learning open-vocabulary physical skill. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 15781–15790, 2025.
- Di Carlo, J., Wensing, P. M., Katz, B., Bledt, G., and Kim, S. Dynamic locomotion in the mit cheetah 3 through convex model-predictive control. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1–9, 2018. doi: 10.1109/IROS.2018.8594448.
- Eckert, P., Sprwitz, A., Witte, H., and Ijspeert, A. J. Comparing the effect of different spine and leg designs for a small bounding quadruped robot. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3128–3133, 2015. doi: 10.1109/ICRA.2015.7139629.
- Grandia, R., Farshidian, F., Ranftl, R., and Hutter, M. Feed-back mpc for torque-controlled legged robots. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4730–4737. IEEE, 2019.
- He, T., Luo, Z., Xiao, W., Zhang, C., Kitani, K., Liu, C., and Shi, G. Learning human-to-humanoid real-time whole-body teleoperation. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8944–8951. IEEE, 2024.
- Howard, T. M., Tellex, S., and Roy, N. A natural language planner interface for mobile manipulators. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6652–6659, 2014. doi: 10.1109/ICRA.2014.6907841.
- Ji, M., Peng, X., Liu, F., Li, J., Yang, G., Cheng, X., and Wang, X. Exbody2: Advanced expressive humanoid whole-body control. *arXiv preprint arXiv:2412.13196*, 2024.
- Jian, P., Wei, X., Liu, Y., Moore, S. A., Zavlanos, M. M., and Chen, B. Lapp: Large language model feedback for preference-driven reinforcement learning. *arXiv preprint arXiv:2504.15472*, 2025.
- Kress-Gazit, H., Fainekos, G. E., and Pappas, G. J. Translating structured english to robot controllers. *Advanced Robotics*, 22(12):1343–1359, 2008. doi: 10.1163/156855308X344864.
- Kumar, A., Fu, Z., Pathak, D., and Malik, J. Rma: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.
- Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., and Hutter, M. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):eabc5986, 2020.
- Li, H., Zhang, T., Yu, W., and Wensing, P. M. Versatile real-time motion synthesis via kino-dynamic mpc with hybrid-systems ddp. *arXiv preprint arXiv:2209.14138*, 2022.
- Li, Y., Li, J., Fu, W., and Wu, Y. Learning agile bipedal motions on a quadrupedal robot. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9735–9742. IEEE, 2024.
- Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., and Zeng, A. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022.
- Lin, K., Agia, C., Migimatsu, T., Pavone, M., and Bohg, J. Text2motion: From natural language instructions to feasible plans. *Autonomous Robots*, 47(8):1345–1365, 2023.
- Long, J., Wang, Z., Li, Q., Gao, J., Cao, L., and Pang, J. Hybrid internal model: Learning agile legged locomotion with simulated robot response. *arXiv preprint arXiv:2312.11460*, 2023.

- Luo, Z., Cao, J., Merel, J., Winkler, A., Huang, J., Kitani, K., and Xu, W. Universal humanoid motion representations for physics-based control. *arXiv preprint arXiv:2310.04582*, 2023.
- Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., and Anandkumar, A. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023.
- Mahankali, S., Lee, C.-C., Margolis, G. B., Hong, Z.-W., and Agrawal, P. Maximizing quadruped velocity by minimizing energy. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11467–11473, 2024. doi: 10.1109/ICRA57147.2024.10609983.
- Mahmood, N., Ghorbani, N., Troje, N. F., Pons-Moll, G., and Black, M. J. AMASS: Archive of motion capture as surface shapes. In *International Conference on Computer Vision*, pp. 5442–5451, October 2019.
- Margolis, G. B. and Agrawal, P. Walk these ways: Tuning robot control for generalization with multiplicity of behavior. In *Conference on Robot Learning*, pp. 22–31. PMLR, 2023.
- Marhefka, D., Orin, D., Schmiedeler, J., and Waldron, K. Intelligent control of quadruped gallops. *IEEE/ASME Transactions on Mechatronics*, 8(4):446–456, 2003. doi: 10.1109/TMECH.2003.820001.
- Mees, O., Hermann, L., and Burgard, W. What matters in language conditioned robotic imitation learning over unstructured data. *IEEE Robotics and Automation Letters*, 7(4):11205–11212, 2022.
- Nahrendra, I., Yu, B., and Myung, H. Dreamwaq: Learning robust quadrupedal locomotion with implicit terrain imagination via deep reinforcement learning. *arXiv preprint arXiv:2301.10602*, 2023.
- Ouyang, Y., Li, J., Li, Y., Li, Z., Yu, C., Sreenath, K., and Wu, Y. Long-horizon locomotion and manipulation on a quadrupedal robot with large language models. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 11157–11164. IEEE, 2025.
- Peng, X. B., Abbeel, P., Levine, S., and Van de Panne, M. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions On Graphics (TOG)*, 37(4):1–14, 2018a.
- Peng, X. B., Kanazawa, A., Malik, J., Abbeel, P., and Levine, S. Sfv: Reinforcement learning of physical skills from videos. *ACM Transactions On Graphics (TOG)*, 37(6):1–14, 2018b.
- Peng, X. B., Ma, Z., Abbeel, P., Levine, S., and Kanazawa, A. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Transactions on Graphics (ToG)*, 40(4):1–20, 2021.
- Qiu, R.-Z., Song, Y., Peng, X., Suryadevara, S. A., Yang, G., Liu, M., Ji, M., Jia, C., Yang, R., Zou, X., et al. Wildlma: Long horizon loco-manipulation in the wild. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 10011–10019. IEEE, 2025.
- Raibert, M. H. Trotting, pacing and bounding by a quadruped robot. *Journal of Biomechanics*, 23:79–98, 1990. ISSN 0021-9290. doi: [https://doi.org/10.1016/0021-9290\(90\)90043-3](https://doi.org/10.1016/0021-9290(90)90043-3). URL <https://www.sciencedirect.com/science/article/pii/0021929090900433>. International Society of Biomechanics.
- Shridhar, M., Manuelli, L., and Fox, D. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, pp. 894–906. PMLR, 2022.
- Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., and Garg, A. Prog-prompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022.
- Smith, L., Kew, J. C., Li, T., Luu, L., Peng, X. B., Ha, S., Tan, J., and Levine, S. Learning and adapting agile locomotion skills by transferring experience. *arXiv preprint arXiv:2304.09834*, 2023.
- Stepputtis, S., Campbell, J., Phielipp, M., Lee, S., Baral, C., and Ben Amor, H. Language-conditioned imitation learning for robot manipulation tasks. *Advances in Neural Information Processing Systems*, 33:13139–13150, 2020.
- Tang, Y., Yu, W., Tan, J., Zen, H., Faust, A., and Harada, T. Saytap: Language to quadrupedal locomotion. *arXiv preprint arXiv:2306.07580*, 2023.
- Tessler, C., Kasten, Y., Guo, Y., Mannor, S., Chechik, G., and Peng, X. B. Calm: Conditional adversarial latent models for directable virtual characters. In *ACM SIGGRAPH 2023 Conference Proceedings*, SIGGRAPH ’23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701597. doi: 10.1145/3588432.3591541. URL <https://doi.org/10.1145/3588432.3591541>.
- Tirinzoni, A., Touati, A., Farebrother, J., Guzek, M., Kanervisto, A., Xu, Y., Lazaric, A., and Pirodda, M. Zero-shot whole-body humanoid control via behavioral foundation models. *arXiv preprint arXiv:2504.11054*, 2025.

- Tirumala, S., Sagi, A., Paigwar, K., Joglekar, A., Bhatnagar, S., Ghosal, A., Amrutur, B., and Kolathaya, S. Gait library synthesis for quadruped robots via augmented random search. *arXiv preprint arXiv:1912.12907*, 2019.
- Vemprala, S. H., Bonatti, R., Bucker, A., and Kapoor, A. Chatgpt for robotics: Design principles and model abilities. *Ieee Access*, 12:55682–55696, 2024.
- Villarreal, O., Barasuol, V., Wensing, P. M., Caldwell, D. G., and Semini, C. Mpc-based controller with terrain insight for dynamic legged locomotion. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2436–2442. IEEE, 2020.
- Winkler, A. W., Bellicoso, C. D., Hutter, M., and Buchli, J. Gait and trajectory optimization for legged systems through phase-based end-effector parameterization. *IEEE Robotics and Automation Letters*, 3(3):1560–1567, 2018. doi: 10.1109/LRA.2018.2798285.
- Xu, S., Ling, H. Y., Wang, Y.-X., and Gui, L.-Y. Intermimic: Towards universal whole-body control for physics-based human-object interactions. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 12266–12277, 2025.
- Yu, R., Park, H., and Lee, J. Human dynamics from monocular video with dynamic camera movements. *ACM Trans. Graph.*, 40(6), December 2021. ISSN 0730-0301. doi: 10.1145/3478513.3480504. URL <https://doi.org/10.1145/3478513.3480504>.
- Yu, W., Gileadi, N., Fu, C., Kirmani, S., Lee, K.-H., Arenas, M. G., Chiang, H.-T. L., Erez, T., Hasenclever, L., Humplik, J., et al. Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647*, 2023.
- Zhang, Z., Bashkurov, S., Yang, D., Taylor, M., and Peng, X. B. Add: Physics-based motion imitation with adversarial differential discriminators. *arXiv preprint arXiv:2505.04961*, 2025.
- Zhou, K., Mu, Y., Song, H., Zeng, Y., Wu, P., Gao, H., and Liu, C. Adaptive interactive navigation of quadruped robots using large language models. *arXiv preprint arXiv:2503.22942*, 2025.

A. Implementation Details

We present implementation details for our baselines below.

Eureka uses Claude Sonnet 4.5 to design reward functions using evolutionary program synthesis. Eureka takes as input a natural language task description ℓ , the environment definition $\mathcal{M}$ expressed as executable source code exposing state and action variables, a black-box task fitness function F , and a reinforcement learning algorithm $\mathcal{A}$ used to optimize candidate rewards. The environment source code $\mathcal{M}$ is parsed to extract only the components that define observations, actions, and relevant state variables. This code, together with the task description ℓ and generic reward-design instructions, is provided as context to the LLM. No task-specific prompts or reward templates are used.

Given the contextualized prompt, the LLM generates a batch of K executable reward programs

$$\{R_1, R_2, \dots, R_K\},$$

each implemented as a TorchScript-compatible function returning a scalar reward and a dictionary of interpretable reward components. Each reward candidate R_k is evaluated by training a policy

$$\pi_k = \mathcal{A}(\mathcal{M}, R_k)$$

using RL. The resulting policy is scored using the task fitness function $F(\pi_k)$, yielding a scalar fitness value s_k . The best-performing reward $R^* = \arg \max_k s_k$ is retained as the current best reward. If s^* exceeds the best score achieved so far, the best reward is updated. Training statistics, including success rates, episode lengths, and per-component reward magnitudes, are summarized into a textual reward reflection. This reflection provides diagnostic feedback describing which reward components are ineffective, poorly scaled, or dominant. The best reward R^* and its reflection are appended to the LLM context. The LLM is then prompted to generate a new batch of mutated reward candidates by modifying the structure, scaling, or functional forms of existing components, or by introducing new components. This defines one evolutionary iteration. The processes of sampling, evaluation, reflection, and mutation are repeated for N iterations, optionally with multiple random restarts to mitigate local optima. The final output is the reward program with the highest observed fitness across all iterations. The algorithm returns a fully executable, human-interpretable reward function that can be reused for policy training, curriculum learning, or further refinement with human feedback.

MimicAgent orchestrates many task-specific sub-agents to generate kinematically feasible quadruped reference trajectories. For parity with Eureka, all MimicAgent sub-agents use Claude Sonnet 4.5. We implement MimicAgent using

LangGraph, a graph-based framework that represents an agentic system as a directed graph in which nodes correspond to agents or deterministic processing steps, edges define control flow and conditional transitions, state is a shared, and persistent objects are passed between nodes. Unlike linear prompting pipelines, LangGraph enables cyclic execution (e.g., refinement loops), conditional branching based on evaluation outcomes, and the accumulation of long-horizon context without prompt bloat. This structure is essential for MimicAgent, as trajectory synthesis requires iterative correction based on downstream validation.

To generate a diverse set of candidate skills, we employ a dedicated skill proposal agent. Given an initial prompt, the agent generates K skill descriptions per iteration. To prevent duplication and encourage diversity, all previously generated skills are appended to the prompt context in subsequent iterations. The agent is instructed to propose novel behaviors that differ in contact patterns, base motion (translation, rotation, or vertical motion), limb coordination, and stylistic or agility characteristics. This process is repeated until a target number of unique skills is reached. Once generated, each skill description is processed independently by MimicAgent. After generating skill descriptions, MimicAgent processes each description using a skill filter node, motion planner agent, coder agent, code execution agent, task-agnostic unit test node, motion failure diagnosis agent, and a code rewrite agent. A shared state object accumulates all intermediate artifacts, enabling coherent multi-step refinement. We describe each component below.

First, the skill filter node performs lightweight semantic validation of the natural-language skill description. Its purpose is to reject skills that are fundamentally underspecified, contradictory, or kinematically impossible even under idealized assumptions. Filtering at this stage prevents downstream agents from wasting tokens on faulty specifications. The motion planner agent then converts a skill description into a structured phase-based motion plan. Its prompt enforces a strict planning-only role, and explicitly instructs the LLM to avoid code generation, equations, or hyperparameter selection. All motion is described using a single cyclic phase variable $\phi \in [0, 1]$. The output is a single JSON object with a fixed schema, including, phase decomposition into sub-phases, per-phase contact expectations, base velocity and angular rate intent, qualitative leg trajectory descriptions, and declarative constraints aligned with unit tests. Importantly, the planner specifies intent rather than implementation, enabling multiple realizations of the same skill structure.

Next, the coder agent translates the structured motion plan into executable Python code that implements a subclass of `BaseMotionGenerator`. The agent must precisely adhere to the provided base class interfaces and architectural

patterns, and is explicitly prohibited from modifying base class APIs, inventing new abstractions, and emitting partial code or placeholders. All base motion is implemented via velocity and angular rate commands, while all leg trajectories are generated as phase-continuous functions in the body frame. Few-shot reference implementations (e.g., a trot gait) are provided to demonstrate correct phase handling, swing-stance decomposition, and trajectory shaping. These references guide code structure without constraining motion semantics. After the coder agent implements executable Python code, the code executor agent validates it by executing the code in the MuJoCo simulator and provides a binary signal based on actual runtime behavior. Unlike the planning or critique agents, the code executor agent does not analyze or modify code. If the simulator terminates successfully without runtime errors, the agent records a success flag in the shared state. If execution fails, the agent captures and propagates the standard error output, exception trace, and return code. These signals are passed verbatim to downstream agents for critique or refinement.

Generated trajectories are evaluated using a suite of task-agnostic unit tests that detect structural issues without encoding task success. These tests identify issues such as ground penetration, joint limit violations, excessive air time, and unsafe base height envelopes. Each test produces a scalar score, and trajectories must exceed a minimum threshold to pass validation. If validation fails, the motion failure diagnosis agent analyzes the trajectory using the current skill implementation, unit test feedback, and original skill description. Importantly, this agent does not modify code, and is instead instructed to produce a structured diagnostic report identifying the most likely root cause of each failure.

Lastly, the code rewrite agent revises the trajectory-generating code based on unit test feedback and diagnostic analysis. Its prompt enforces a minimal-change philosophy: only components responsible for reported failures may be modified. Structural failures (e.g., joint violations) are addressed first by adjusting amplitudes, phase timing, or trajectory envelopes. Quality failures (e.g., velocity spikes) are addressed through smoothing and phase-continuous reshaping without flattening the motion or altering its defining characteristics. The agent outputs a single, complete Python file, closing the agentic loop. This execution loop repeats until either the generated trajectory passes all unit tests, or a fixed iteration budget is reached. All intermediate artifacts are retained in the shared state, enabling consistent multi-step reasoning.

B. Training Time and Token Analysis

In Table 4, we show that MimicAgent uses significantly fewer tokens than Eureka. We posit that this is likely because Eureka spends many tokens in its massively parallel

reward exploration. In contrast, MimicAgent uses significantly fewer tokens due to the compactness of LangGraph’s state management. Somewhat surprisingly, training policies with Eureka is considerably faster than training DeepMimic policies with MimicAgent’s reference trajectories. Specifically, DeepMimic requires at least 15,000 PPO iterations, taking roughly 12 hours on an NVIDIA RTX 3090 to achieve satisfactory tracking performance. In contrast, Eureka, configured with 3,000 PPO iterations, 5 reward iterations, and 16 reward samples per iteration trains in approximately 6 hours.

Table 4. Token Usage Between Baselines. We compare the number of tokens used by both MimicAgent and Eureka below. Although MimicAgent iteratively refines generated trajectories and has multiple sub-agents, Eureka uses significantly more tokens across all five baseline tasks due to its parallel reward generation approach. Lower is better.

Skill	Walking	Bounding	Side Flip	Front Flip	Aerial Crossover
Robot	Go2	Go2	Go2-W	Go2-W	Go2-W
Eureka	803K	917K	676K	682K	730K
MimicAgent	14K	12K	13K	36K	16K

C. Transferability to Different Imitation RL Frameworks

We evaluate the transferability of MimicAgent-generated trajectories across different imitation learning frameworks in Table 5. In particular, we compare the DeepMimic (Peng et al., 2018a) with ADD (Zhang et al., 2025).

ADD achieves consistently lower tracking errors at both the root and joint levels, aligning with its reported improvements in tracking accuracy. However, despite these quantitative gains, we observe that policies trained with DeepMimic produce more natural and stable motions. In contrast, ADD trained policies tend to exhibit less natural motion characteristics. These results highlight a trade-off between strict tracking accuracy and natural movement, suggesting that lower tracking error does not necessarily translate to more realistic behaviors in practice.

Table 5. Tracking Error Averaged Across All Skills. We report root level(RP, RR, RLV, RAV) and joint level (JP, JV) mean tracking errors averaged across all the 5 skills described in the main paper. Although ADD achieves lower tracking error compared to DeepMimic, we find that resulting policies are qualitatively worse.

Method	RP	RR	JR	JV	RLV	RAV
DeepMimic (Ours)	0.056	0.1102	0.3232	5.0314	0.1664	0.8612
ADD	0.1086	0.0990	0.1644	1.7587	0.1961	0.7208

D. Evaluating More Difficult Skills

We evaluate two skills that are difficult to keyframe below.

- *UniTree Go2-W Reverberating Yaw Pulse (RYP)* A oscillatory motion with decreasing amplitude envelope,

mimicking physical damping. Phase controls the amplitude decay and frequency of reverberating angular pulses. Unlike constant-rate spinning, this creates visually distinct accelerating-decelerating rhythm.

- *UniTree Go2-W Crab Diagonal Scuttle (CDS)* The robot scuttles diagonally with body axis perpendicular to travel vector. This decouples body orientation from travel direction, creating crab-like diagonal motion. Phase coordinates cross-body leg sweeps where front and rear act in opposition. The orientation remains perpendicular to the velocity vector.

In Table 6 and Table 7, we compare the time-to-annotation and tracking accuracy for these two more difficult skills. Notably, manually keyframing both skills takes considerably longer than MimicAgent, supporting our hypothesis that as skill complexity increases, human annotation time will grow significantly faster than MimicAgent skill generation time. We also find that policies trained with MimicAgent trajectories more faithfully represent the reference trajectory than those trained with keyframed trajectories, likely because MimicAgent’s trajectories are more dynamically feasible than human annotations.

Table 6. Comparison of Time-to-Annotation. We evaluate the time to annotation (in minutes) between human annotated keyframes and MimicAgent keyframes. Notably, MimicAgent is consistently faster than human annotated keyframes on the two hard skills. Lower is better.

Skill Robot	CDS Go2-W	RYP Go2-W
Key Frames	14.51	12.37
MimicAgent	1.59	1.81

E. Transferability to Different Simulators

We train DeepMimic policies using only trajectory tracking rewards, without any domain randomization in Isaac Gym. The trained policies are then evaluated in both Isaac Gym and Newton. As shown in Table 8, we find that our DeepMimic policies achieve similar error rates in both simulators, suggesting that our trained policies are robust to simulator characteristics. Further, this suggests that such policies may transfer well to on-robot deployment in the real world.

Table 8. Cross-Simulator Generalization. We evaluate Isaac Gym trained policies in Newton. We observe that policies achieve similar performance across simulators, highlighting policy robustness. Metrics are averaged across all skills. Lower is better.

Simulator	E_{RP}	E_{RR}	E_{JR}	E_{JV}	E_{RLV}	E_{RAV}
IsaacGym	0.0640	0.1331	0.3504	5.1658	0.1799	0.9936
Newton	0.0592	0.2182	0.4320	5.5619	0.1792	1.0971

Table 7. Comparison with Reference Motion. We evaluate DeepMimic policies trained with human annotated keyframes and MimicAgent keyframes against the reference trajectories. Across all skills and metrics, MimicAgent policies achieve lower mean squared error on both pose-level errors (root and body position and rotation), and motion-level errors (linear and angular velocities). Lower is better.

Method Robot	Error Metric	CDS Go2-W	RYP Go2-W
Key Frames	E_{RP}	0.044	0.041
	E_{RR}	0.201	0.142
	E_{JR}	0.033	0.025
	E_{JV}	6.171	4.721
	E_{RLV}	0.261	0.126
	E_{RAV}	0.797	1.172
MimicAgent	E_{RP}	0.0348	0.0129
	E_{RR}	0.0322	0.0885
	E_{JR}	0.0093	0.0178
	E_{JV}	1.6085	2.3479
	E_{RLV}	0.1241	0.0474
	E_{RAV}	0.2639	0.4547

F. Impact of Unit Tests on Success Rate

We ablate the impact of task-agnostic unit tests on the overall success rate of trajectories generated by MimicAgent in Table 9. To isolate the contribution of different constraints, we define six progressive unit test configurations: no unit tests (C0), adding a base height limit (C1), adding joint limit checks (C2), adding a feet airtime penalty (C3), adding ground penetration checks (C4, default used in the main paper), and adding dynamically generated, skill-specific unit tests proposed by an LLM (C5). To evaluate these configurations, we randomly sample five skills from the 101 feasible skills generated by our skill proposal agent (Figure 4). For each skill and unit test configuration, we manually validate (e.g. pass / fail) whether the executed skill closely matches the text description. The final success metric for each configuration is the total success rate across the five evaluated skills.

Our results show that progressively adding unit tests improves the overall success rate, with C4 and C5 achieving the highest policy success rate of 80%. The dip observed at C3 can be explained by an interaction between constraints. Specifically, enforcing feet airtime in isolation successfully detects when all feet leave the ground, but compensates by driving the body downward to restore contact. Without a ground penetration constraint, this leads to feet clipping into the terrain. This issue is fully resolved at C4, where the ground penetration constraint is introduced alongside the airtime constraint, demonstrating that these two constraints are complementary and must be applied together to be effective. At C5, LLM-proposed tests introduce skill-specific

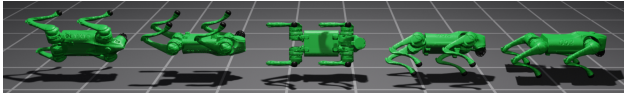
Table 9. Unit Test Ablation. We evaluate the impact of unit tests on the quality of reference trajectories and learned policies. Checkmarks indicate active unit tests for each configuration. Adding all unit tests yields the best performance.

Config	Height	Joint	Airtime	Ground	LLM	Reference	Policy
C0	×	×	×	×	×	1/5	1/5
C1	✓	×	×	×	×	3/5	2/5
C2	✓	✓	×	×	×	4/5	3/5
C3	✓	✓	✓	×	×	4/5	2/5
C4	✓	✓	✓	✓	×	5/5	4/5
C5	✓	✓	✓	✓	✓	5/5	4/5

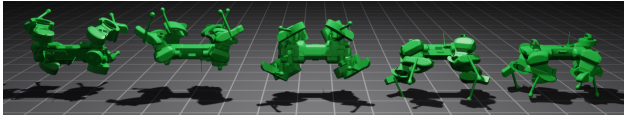
constraints tailored to the nuances of each motion, resulting in a modest but consistent improvement in the quality of reference trajectories.

G. Additional Quadruped Morphologies

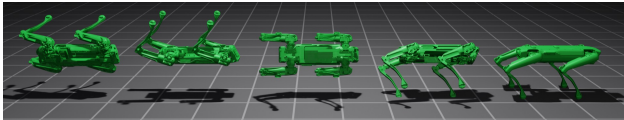
To demonstrate the generalization capabilities of our approach, we deploy the same LLM-generated skill code originally designed for the Go2 across different quadruped morphologies, *without any prompt modifications*. Since MimicAgent generates motion plans using a task-agnostic, phase-based formulation combined with body-frame kinematic commands, the resulting Python implementations remain independent of specific robot dimensions and mass properties. As shown in Figure 7, a single generated skill can therefore be applied directly to produce kinematically valid reference trajectories for a variety of distinct quadruped platforms.



(a) Go2 Reference Trajectory



(b) ANYmal Reference Trajectory



(c) Spot Reference Trajectory

Figure 7. Morphology Generalization. The same LLM-generated skill code is applied directly to (a) Go2, (b) ANYmal C, and (c) Spot *without any prompt modifications*. Each row shows keyframes from the resulting reference trajectories.

H. Keyframe Ablation on Reference Quality

In Table 10, we analyze the impact of keyframe quality on tracking performance. To do this, we construct lower-quality trajectories by subsampling the manually created keyframes. Notably, tracking performance remains comparable across

Table 10. Comparison Between High Quality and Low Quality Keyframes We train policies on five skills with manual keyframes of 133 (high-quality) and 26 (low-quality) frames. We find that DeepMimic achieves similar performance in both cases. Lower is better.

Quality	E_{RP}	E_{RR}	E_{JR}	E_{JV}	E_{RLV}	E_{RAV}
HQ	0.1128	0.1712	0.4134	7.7134	0.2670	0.9354
LQ	0.0973	0.1958	0.41765	7.0548	0.2361	1.0719

both the original and degraded trajectories, indicating that DeepMimic is robust to sparse and noisy keyframes. This robustness helps explain why policies learned from human-provided keyframes and those generated by MimicAgent achieve similar performance.

I. Baseline Skills Prompts

We present the prompts used to generate the five skills used in our user study below.

Skill Prompt (Bounding Gait).

Control the Go2 robot to execute a dynamic bounding gait while tracking commanded forward velocities in the x-direction. The robot alternates between synchronized front-leg contacts and synchronized rear-leg contacts, with ballistic flight phases between each contact where all feet leave the ground. The robot maintains a forward-facing orientation with minimal roll and yaw, allows natural pitch oscillation during flight and landing, enforces left-right force symmetry within each leg pair to prevent lateral drift, and avoids any four-foot stance phases. Bounding should generate sufficient vertical impulse for observable airborne time, maintain consistent body height, track the commanded forward velocity smoothly, and transition to a stable standing posture when commanded velocities are near zero. The gait remains rhythmic, energy-efficient, and robust to perturbations across varying speeds.

Skill Prompt (Front Flip).

To make the quadruped robot to execute a front flip (forward somersault) conditioned on the commanded forward velocity. When the commanded forward velocity v_x is strictly positive, the robot should crouch to build energy, explosively launch upward while initiating forward pitch rotation, complete a full 360-degree rotation around its lateral (pitch) axis while airborne, and land stably on all four feet in an upright orientation with minimal post-landing motion. The flip should be performed approximately in place with minimal horizontal drift, the rotation should be clean without excessive roll or yaw deviation, and the robot must maintain sufficient height throughout the airborne phase to complete the rotation before touchdown. When the commanded forward velocity v_x is zero or negative, the robot should remain idle in a stable standing posture with all feet on the ground, minimal base motion, low angular velocities, and low actuation effort. The robot should ignore lateral velocity and yaw commands, prioritizing explosive launch power, fast rotational velocity during flight, proper timing to achieve upright orientation before landing, and controlled impact absorption for stable recovery after touchdown.

Skill Prompt (Trotting Gait).

to make the quadruped robot execute a stable and rhythmic trot gait while accurately tracking randomly sampled forward linear velocity commands along the x-axis. The robot should move using synchronized diagonal leg pairs that alternate between push-off and landing phases, producing brief and consistent aerial phases between contacts. Maintain a forward-facing torso with minimal roll, yaw and pitch of a trot, and generate steady, efficient forward propulsion in the commanded direction. The gait should remain robust to rapid velocity changes, minimize foot scuffing and lateral slip, recover gracefully from disturbances, and produce a fast, natural, and agile trotting motion suitable for dynamic quadrupedal locomotion.

Skill Prompt (Aerial Crossover).

When commanded with positive forward velocity (v_x), the wheeled-legged robot should carve forward smoothly on its wheels while briefly lifting alternating diagonal leg pairs into the air. These rapid crossover wave motions of the legs occur without breaking wheel contact and are used to actively reshape the robots inertia and base orientation. The legs must re-enter contact in phase, allowing the robot to transition seamlessly back into a stable skating wave with increased forward speed and agile directional response. This skill treats leg motion as an inertial control surface rather than a support requirement, rewarding fast, rhythmic aerial leg alternation that enhances forward locomotion stability and agility. When commanded with zero or negative forward velocity (v_x) and random v_y or yaw rate, the robot should remain stationary in a stable, seated posture.

Skill Prompt (Side Flip).

Control the Go2w robot to perform a dynamic side flip conditioned on the commanded forward velocity. When the commanded forward velocity v_x is strictly positive, the robot should execute a single lateral side flip in place by rotating primarily about its body x-axis (roll), lifting all feet and wheels off the ground briefly, completing approximately one full rotation, and landing stably without falling. The robot should not translate significantly during the flip.

When the commanded forward velocity v_x is zero or negative, the robot should remain idle in a stable, seated posture with minimal base motion, low joint velocities, and low actuation effort.

The robot should ignore v_y and yaw commands. Stability after landing, controlled angular motion, limited base translation, and low unnecessary torque are desired.

J. MimicAgent Sub-Agent Prompts

Skill Generator Agent Prompt

SYSTEM: Skill Generator Agent

Task: Propose atomic motion skills for a quadruped / wheeled-quadruped robot (kinematic, MuJoCo feasible).

Constraints:

- No torques, forces, or numeric speeds.
- Describe only base and limb coordination.
- Skills must be kinematically feasible in MuJoCo.
- ...

Phase: Continuous $[0, 1]$, with multiple sub-phases and internal variation.

Wheel Abstraction: Active vs Passive conceptual interpretations.

Skill Mix:

- Locomotion: 50–60%
- Dynamic in-place: 20–30%
- Stylized / Hybrid: 10–20%
- ...

Output: JSON list of skills, each containing:

- skill_id
- skill_name
- skill_intent
- skill_description
- think (conceptual reasoning)
- ...

FEW-SHOT EXAMPLES: ...

Human Prompt: Generate exactly {skills_per_generation} skills. Follow the schema strictly. Return a JSON list only.

Motion Planner Agent Prompt

SYSTEM: Motion Planner Agent

Task: Take a high-level locomotion or agility skill and produce a structured, phase-based motion plan executable by a BaseMotionGenerator-style controller.

Constraints:

- Do NOT generate code or equations.
- Do generate execution logic, contact reasoning, coordination, and unit-test-compatible constraints.
- Base controller exposes ONLY:
 - set_velocity_command
 - set_angular_velocity_command
 - ...
- ...

Phase: Single variable $[0, 1]$, smooth, monotonic, cyclic. Motion decisions are functions of phase.

Output Format: Strict JSON with keys:

- skill_id
- motion_plan:
 - summary:
- ...

Critical Rules: Do NOT invent APIs, rename keys, use absolute poses, or world-frame paths. Plan contact base motion leg motion. All constraints must map to unit tests.

Mental Model:

1. Visualize robot over one full cycle.
2. ...

FEW-SHOT EXAMPLES: ...

Human Prompt: Generate a JSON motion plan for a given skill. Follow schema strictly. Return JSON only.

Coder Agent Prompt

SYSTEM: Coder Agent

Role: Convert a structured, phase-based kinematic motion specification into executable Python code for MuJoCo animation and data generation.

Inputs Provided:

- Structured motion plan from Motion Planner Agent
- Base class definitions and utilities
- ...

Robot Model Assumptions:

- Legged or wheeled quadruped
- Purely kinematic motion
- ...

Core Responsibility:

- Interpret motion intent and phase structure
- Choose reasonable numeric parameters
- ...

Code Generation Constraints:

- Output exactly one Python file
- No dynamics, forces, or controllers
- No explanations or markdown
- ...

External Context:

Base classes, math utilities, and reference implementations are provided externally.

...

Human Prompt:

- Motion specification provided in structured JSON
- Skill class name:
 {SKILL_NAME}_MotionGenerator
- Leg names: FL, FR, RL, RR
- ...

BEGIN IMPLEMENTATION

Motion Failure Diagnosis Agent Prompt

SYSTEM: Motion Failure Diagnosis Agent

Role: Analyze failures in purely kinematic, phase-based quadruped / wheeled-quadruped motion skills.

You DO:

- Reason about failures and root causes
- Propose corrective strategies

You DO NOT:

- Write or modify code
- Suggest dynamics or control changes

Inputs:

- Current skill implementation
- Evaluation feedback
- Skill description
- ...

Execution Context:

- Purely kinematic
- Phase-driven motion
- MuJoCo for animation only
- ...

Failure Types:

- Structural failures (e.g., penetration, limits)
- Quality failures (e.g., spikes)
- ...

Output Format:

- Summary of failures
- Root cause analysis (per failure)
- Corrective strategies
- ...

Human Input:

{CURRENT_SKILL_CODE}

Code Rewrite Agent Prompt

SYSTEM: Code Rewrite Agent

Role: Rewrite an existing kinematic motion generator so that it passes automated evaluation while preserving the original skill intent.

You Receive:

- Current Python skill implementation
- Unit test feedback
- Skill description
- Motion failure diagnosis
- ...

Execution Context:

- Purely kinematic quadruped / wheeled-quadruped
- Phase-based, continuous motion
- MuJoCo used only for animation
- ...

Failure Modes Handled:

- Structural violations (hard constraints)
- Smoothness and quality violations
- ...

Semantic Constraints:

- Preserve defining characteristics of the skill
- Maintain phase ordering and intent
- ...

Hard Constraints:

- Output a single Python file only
- No explanations or markdown
- No new APIs or class changes
- ...

Human Input:

{CURRENT_SKILL_CODE} ...